SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks:50**

Mode: In Class **Hours : 60 mts**

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I BANDARI LAKSHITHA SANKAR REDDY (192571085) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	**/ 20**	**/ 25**	**/ 20**	**/ 15**	**/ 100**

Annexure A

ANALYTICAL PROBLEM SOLVING

1	Trace the processing of the input expression P = (a * b) + (b * c) * 2 through all phases of a compiler. a) Identify and list all tokens generated during lexical analysis. b) Construct the syntax tree during the parsing phase. c) Generate the intermediate code for the given expression. d) Apply suitable optimizations to the intermediate code. e) Explain the significance of the final target code prod
2	For each of the following Regular Expressions, construct the language. Write any FIVE valid strings for each Regular Expression. a)$(\varepsilon+aa^*)^*$ b)$(a^*+b^*)^*\ abb$ c)$(a^*b^*)^*\ aba\ (a^*+b^*)^*$ d)$(a+ab)^*$ e) $(aba)^*+(a^*b^*)^*$

3	i)Find the number of tokens in the following C statement is
	main( )
	{
	int a=10;
	char b =”abc”;
	in t c = 30;
	ch ar d = “xyz”;
	in /* comment */ t m = 40.5;
	}
	ii)Identify the lexical errors in the following C statement is
	void main()
	{
	int x=10, y=20;
	char * a;
	a= &x;
	x= 1xab;
	}

4	**Find a regular expression corresponding to each of the following subsets of $\{0,1\}^*$:** i) The language of all strings that have an even number of 0’s. ii) The language of all strings that contain at least one 1. iii) The language of all strings in which both the number of 0’s and the number of 1’s are odd. iv) The language of all strings that start with 1 and end with 0. v) The language of all strings that do not contain consecutive 1’s.

5	**Consider a lexical analyzer for a simplified programming language. The language consists of the following tokens:** 1. Keywords: if, else, while, return, int, float 2. Operators: +, -, *, /, =, ==, <, >= 3. Separators: (,), {, }, ;, , 4. Identifiers: A sequence of letters followed by a sequence of digits, e.g., var123 5. Integer Literals: A sequence of digits, e.g., 12345 6. Floating-point Literals: A sequence of digits followed by a dot and another sequence of digits, e.g., 123.45 **The lexical analyzer needs to tokenize the following input string:** int var123 = 12345; float var456 = 123.45; if (var123 == var456) { return; } **Questions:** a) How many tokens are there in the given input string? b) Provide a list of tokens categorized by their types (keywords, operators, separators, identifiers, integer literals, floating-point literals). c) Assuming a finite state machine (FSM) is used for lexical analysis, estimate the number of state transitions required to tokenize the given input string. Provide your reasoning based on the transitions needed for each token type.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method

Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

NAME : BANDARI LAKSHITHA SANKAR REDDY
REGNO : 19257 1085
CODE : CSA 1409 - SLOT C
COURSE : COMPILER DESIGN

CO1

Analytical Problem solving

1. $P = (a*b) + (b*c)*2$

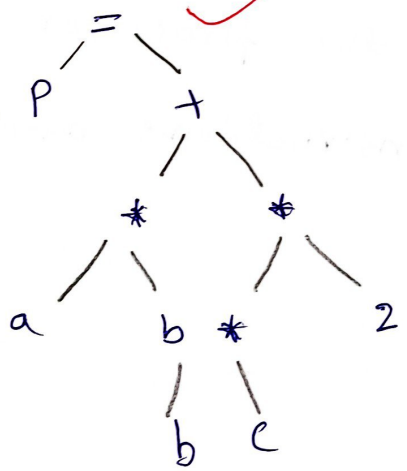
(a) Tokens are generated during lexical analysis

P → Identifier
= → Assignment operator
(→ left Parenthesis
a → Identifier
* → Multiplication operator
b → Identifier
) → right parenthesis
+ → Addition operator
(→ Left Parenthesis
b → Identifier
* → Multiplication operator
c → Identifier
) → Right parenthesis
* → Multiplication operator
2 → Integer Constant.

Total number of tokens = 15

(b) Syntax tree

P = (a*b) + ((b*c)*2)



89/100

03/10/26

(c) Intermediate Code

Using three-address code:

```
t1 = a * b
t2 = b * c
t3 = t2 * 2
t4 = t1 + t3
P = t4
```

d) optimization

The unnecessary temporary t_4 can be removed:

```
t1 = a * b
t2 = b * c
t3 = t2 * 2
P = t1 + t3
```

The number of unnecessary temporary operations is reduced.

e) Final target Code

```
LOAD R1, a
MUL R1, b
LOAD R2, b
MUL R2, c
MUL R2, 2
ADD R1, R2
STORE P, R1
```

The target code is the final machine-level representation that can be executed by the processor. Optimization helps reduce unnecessary instructions and improves execution efficiency.

) $(\varepsilon + aa^*)^*$

The expression generates strings containing only a.

Valid strings: ε, a, aa, aaa, aaaa

(b) $(a^* + b^*)^* abb$

The required part abb must occur.

Valid strings: abb, aabb, aaabb, babb, ababb

This required.

(c) $(a^* b^*)^* aba (a^* + b^*)^*$

Valid strings: aba, aaba, ababa, abaa, abab

(d) $(a+ab)^*$

Each repeated part can be either a or ab.

Valed strings:

ε, a, ab, aa, aab

(e) $(aba)^* + (a^* b^*)^*$

Valid strings are:

ε, aba, abaaba, a, b

3. (i) Find the number of tokens in the given c statement.

```
main()
{
int a=10;
char b="abc";
int c=30;
char d = "xyz";
in /* comment */ tm= 40.5;
}
```

The comment /* comment */ is ignored by the lexical analyzer.

Token Count:

```
main ( )      → 3
{             → 1
int a= 10 ;   → 5
char b ="abc" ; → 5
int  c = 30 ;  → 6
char d = "xyz" ; → 6
int m = 40.5 → 6
}             → 1
```

[3 + 1 + 5 + 5 + 6 + 6 + 6 + 1].

Number of tokens : 33

Note that int and char are treated as seperate tokens because spaces occur between them.

(ii) Identify the lexical errors

```
Void main ( )
{
    int x = 10, y = 20;
    Char * a ;
    a = &x ;
    x = 1xab ;
}
```

The lexical error is 1xab - It is not a valid integer literal because alphabetic characters occur after the initial digit.

refore, the lexical error is the invalid numerical literal 1xab.

Find a regular expression

(i) Strings having an even number of 0's

$1^*(01^*01^*)^*$

(ii) The language of all strings that contain at least one 1.

$(0+1)^*1(0+1)^*$

(iii) The language of all strings in which both the number of 0's and the number of 1's are odd.

$(00+11)^*((01+10)((00+11)^*(01+10)(00+11)^*(01+10)))^*(00+11)^*$

(iv) the language of all strings that start with 1 and end with 0.

$1(0+1)^*0$

(v) The language of all strings that do not contain consecutive 1's.

$0^*(10^*)^*$

5 Lexical Analyzer.

Given input :

Int var123 = 12345 ; float var456 = 123.45 ; if (var123 == var456) {return;}

(a) Number of tokens

The tokens are :

1. int
2. var123
3. =
4. 12345
5. ;
6. float
7. var456
8. =
9. 123.45
10. ;
11. if
12. (
13. var123
14. ==
15. var456
16.)
17. {
18. return
19. ;
20. }

Number of tokens are 20

(b) Tokens categorized by type

Keywords - int, float, if, return

Identifiers - var123, var456, var123, var456

Operators - =, =, ==

Separators - ;, ;, (,), {, ;, }

Integer literals - 12345

Floating-point literals - 123.45

Ok

Total = 4 + 4 + 3 + 7 + 1 + 1

= 20

FSM State transitions

considering one transition for each character belonging to a token:

Token	characters
int	3
var123	6
=	1
12345	5
;	1
float	5
var456	6
=	1
123.45	6
;	1
if	2
(	1
var123	6
==	2
var456	6
==	2
var456	6
)	1
{	1
return	6
;	1
}	1

Total = 3+6+1+5+1+5+6+1+6+1+2+1+6+2+6+1+
1+6+1+1

= 62

Therefore, approximately 62 character transitions are required when only token characters are considered. If spaces are also counted, the estimate becomes approximately 76 transitions.